

PHP Programming

TOPIC

- ▶ PHP introduction
- ▶ Installation
- ▶ PHP Syntax
- ▶ Include Files
- ▶ Class and Functions
- ▶ Arrays
- ▶ Forms Handling And Superglobals
- ▶ Sessions, Cookies and Authentication Basics
- ▶ CRUD Operations

PHP Introduction

- ▶ PHP is an acronym for "PHP: Hypertext Preprocessor"
- ▶ PHP is a widely-used, open source server-side scripting language
- ▶ PHP scripts are executed on the server
- ▶ PHP is free to download and use
- ▶ PHP runs on all major platforms (Windows, Linux, Unix, Mac OS X, etc.)
- ▶ PHP is compatible with all leading web servers (Nginx, Apache, Cloudflare, Microsoft IIS, etc.)
- ▶ PHP supports a wide range of databases (MySQL, PostgreSQL, MS SQL, db2, Oracle Database, MongoDB, etc.)

PHP Introduction

- ▶ Before you learning PHP you should have a basic understanding of the following:
 - HTML
 - CSS
 - JavaScript

PHP Installation

- ▶ To start using PHP, you can:
 - Use a web host with PHP and database support
 - Install a web server on your own PC, and then install PHP and a database

PHP Syntax

► Basic PHP Syntax:

- A PHP script is executed on the server, and the plain HTML result is sent back to the browser.
- A PHP script can be placed anywhere in the document.
- A PHP script starts with `<?php` and ends with `?>`:

► The default file extension for PHP files is `.php`.

► A PHP file normally contains some HTML tags and some PHP scripting code.

► PHP Case Sensitivity:

- PHP keywords (e.g. if, else, echo, etc.), classes, functions, and user-defined functions are not case-sensitive.

► However; all variable names are case-sensitive!

► Embedding PHP scripts:

- You can Embed PHP script on HTML, CSS, JavaScript.

PHP Syntax

► PHP echo and print:

- With PHP, there are two basic ways to output data to the screen: **echo** and **print**.

► The differences are small:

- **echo** has no return value, while **print** has a return value of 1 so it can be used in expressions
- **echo** can take multiple parameters, while **print** can take only one argument
- **echo** is marginally faster than **print**

► PHP echo Function

- The **echo** function can be used with or without parentheses: **echo** or **echo()**.
- You can output text can contain **HTML markup**
- You can output **variables**.

PHP Syntax

► PHP print Function

- The **print** function can be used with or without parentheses: **print** or **print()**.
- You can output text can contain **HTML markup**
- You can output **variables**.

► Single or Double Quotes?

- Strings are surrounded by quotes, but there is a difference between single and double quotes in PHP.
- When using double quotes, variables can be inserted to the string as in the example above.
- When using single quotes, variables have to be inserted using the `.` operator

PHP Syntax

► PHP Variables

- Variables are "containers" for storing information.
- A variable can have a short name (like `$x` and `$y`) or a more descriptive name (`$age`, `$carname`, `$total_volume`).

► Rules for PHP variables:

- A variable must start with the `$` sign, followed by the name of the variable
- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and `_`)
- Variable names are case-sensitive (`$age` and `$AGE` are two different variables)
- Remember that PHP variable names are case-sensitive!

PHP Syntax

- ▶ PHP is a Loosely Typed Language
 - PHP automatically associates a data type to the variable, depending on its value.
 - Since the data types are not set in a strict sense, you can do things like adding a string to an integer without causing an error.
 - In PHP 7, type declarations were added. This gives an option to specify the data type expected when declaring a function, and by enabling the strict requirement, which will throw a "Fatal Error" on a type mismatch.
- ▶ PHP supports the following data types:
 - **string** (text values)
 - **int** (whole numbers)
 - **float** (decimal numbers)
 - **bool** (true or false)
 - **array** (multiple values)
 - **object** (stores data as objects)
 - **null** (empty variable)
 - **resource** (references external resources)
 - **mixed** (any value)

PHP Syntax

► PHP Variable Scope:

- PHP variables can be declared anywhere in the PHP code.
- The scope of a variable is the part of the script where the variable can be referenced/used.

► PHP has three different variable scopes:

- global - A variable declared **outside** a function has a GLOBAL SCOPE and can only be accessed outside a function:
- local - A variable declared **within** a function has a LOCAL SCOPE and can only be accessed within that function.
- static - Normally, when a function finishes executing, all of its variables are deleted. However, sometimes we want a local variable NOT to be deleted. We need it for a further job. To achieve this, use the **static** keyword when you first declare the variable. Then, each time the function is called, that variable will have the value from the last time the function was called.

► PHP global Keyword:

- The global keyword is used to access a global variable from within a function.

PHP Syntax

► PHP Type Casting:

- PHP type casting is an explicit process of converting a value from one data type to another, such as from a float to an integer.
- This gives the developer direct control over the data type of a variable.
- You cast a variable by placing the cast type within parentheses before the variable or value you want to convert.

► The PHP casting operators are:

- (string) - Converts to data type String
- (int) - Converts to data type Integer
- (float) - Converts to data type Float
- (bool) - Converts to data type Boolean
- (array) - Converts to data type Array
- (object) - Converts to data type Object
- (unset) - Deprecated. Converts to data type NULL

PHP Syntax

► PHP Constants:

- Constants are like variables, except that once they are defined they cannot be changed or undefined.
- PHP constants can be defined with the `define()` function or the `const` keyword.
- A valid constant name starts with a letter or underscore (no \$ sign before the constant name).

► PHP `define()` Function

- The `define()` function defines a constant at run-time.
- The `define()` function always define constants in the global scope, and can be declared inside conditional blocks (if/else, loops, functions), but they cannot be used to define class constants.

► PHP `const` Keyword

- The `const` keyword defines a constant at compile-time.
- Constants defined with the `const` keyword must be declared at the top-level scope because they are defined at compile-time. This means that they cannot be declared inside functions, loops, if/else statements or try/catch blocks.
- Constants defined with the `const` keyword can be used to define class constants.
- Constants defined with the `const` keyword are case-sensitive.

PHP Include Files

- ▶ PHP include and require Statements
 - The **include** and **require** statements take all the text/code/markup that exists in a file, and inserts it into the file that uses the include/require statement.
 - Including files is very useful for re-using code/text on multiple webpages!
- ▶ The **include** and **require** statements are identical, except upon failure:
 - **include** will produce a warning (E_WARNING) but the script will continue
 - **require** will produce a fatal error (E_ERROR) and the script will stop
- ▶ If you want the execution to go on and show users the output, even if the include file is missing, use the **include** statement. This could be wise for files like headers, footers, or navigation menus.
- ▶ If you want the execution to stop on failure, use the **require** statement. This is wise for configuration files or database connections.

PHP Include Files

► PHP `include_once` Keyword

- The `include_once` keyword is used to embed PHP code from another file. If the file is not found, a warning is shown and the program continues to run. If the file was already included previously, this statement will not include it again.

► PHP `require_once` Keyword

- The `require_once` keyword is used to embed PHP code from another file. If the file is not found, a fatal error is thrown and the program stops. If the file was already included previously, this statement will not include it again.

PHP Include Files

► PHP include vs. require:

- If a file is included with the **include** statement, and PHP cannot find it, the script will continue to execute:
- If a file is included with the **require** statement, and PHP cannot find it, the echo statement will not be executed because the script execution dies after **require** returns a fatal error:
- Use **require** if the include file is required by the application.
- Use **include** if the include file is not required, and the code can continue, even if the include file is not found.

PHP Class and Functions

- ▶ A class is a template for objects, and an object is an instance of class.
- ▶ A "function" is technically called a **method**. It is a block of reusable code that performs a specific task and runs only when it is called.
- ▶ Types of function:
 - Built-in Functions
 - User Defined Functions
- ▶ Create a Function:
 - A user-defined function declaration starts with the keyword **function**, followed by the name of the function.
 - The opening curly brace **{** indicates the beginning of the function code, and the closing curly brace **}** indicates the end of the function.
 - **Note:** A function name is not case-sensitive, and it must start with a letter or an underscore.

PHP Class and Functions

► Passing Arguments by Reference

- Arguments are usually passed by value, which means that a copy of the value is used in the function and the variable that was passed into the function cannot be changed.
- When a function argument is passed by reference, changes to the argument also change the variable that was passed in. To turn a function argument into a reference, use the **&** operator in front of the argument/parameter:

► Variable Number of Parameters

- By using the **...** operator in front of the function parameter, the function accepts an unknown number of parameters. This is also called a variadic function.
- The variadic function argument becomes an array.
- You can only have one argument with variable length, and it has to be the last argument.
- If the variadic argument is not the last argument, you will get an error.

PHP Class and Functions

► PHP is a Loosely Typed Language

- To specify strict mode, we need to set `declare(strict_types=1);`. This must be on the very first line of the PHP file.
- The strict declaration forces things to be used in the intended way.

► PHP Return Type Declarations

- PHP also supports Type Declarations for the return statement.
- To declare a data type for the function return, add a colon (`:`) and the data type right before the opening curly (`{`) bracket when declaring the function.

► PHP - Static Methods

- Static methods can be called directly - without creating an instance of the class first.
- Static methods are declared with the `static` keyword:

PHP Class and Functions

► Define a Class:

- A class is defined by using the **class** keyword, followed by the name of the class and a pair of curly braces ({}). All its properties and methods go inside the braces:
- **Note:** In a class, variables are called properties and functions are called methods!

► Define Objects

- Classes are nothing without objects! We can create multiple objects from a class. Each object has all the properties and methods defined in the class, but they will have different property values.
- Objects of a class are created using the **new** keyword.

► PHP - The \$this Keyword

- The \$this keyword refers to the current object, and is only available inside methods.

PHP Class and Functions

► PHP - Class Constants

- Class constants can be useful if you need to define some constant data within a class.
- A class constant is declared inside a class with the **const** keyword.
- A constant cannot be changed once it is declared.
- Class constants are case-sensitive. However, it is recommended to name the constants in all uppercase letters.
- We can access a constant from outside the class by using the class name followed by the scope resolution operator (**::**) followed by the constant name.
- Or, we can access a constant from inside the class by using the self keyword followed by the scope resolution operator (**::**) followed by the constant name.

► PHP - Static Properties

- Static properties can be called directly - without creating an instance of a class.
- Static properties are declared with the **static** keyword:

PHP Arrays

- ▶ In PHP, an array is a special variable that can hold many values under a single name, and you can access the values by referring to an index number or a name.
- ▶ PHP Array Types
 - **Indexed arrays** - Arrays with a numeric index
 - **Associative arrays** - Arrays with named keys
 - **Multidimensional arrays** - Arrays containing one or more arrays
- ▶ PHP Array Items
 - Array items can be of any data type.
 - The most common are strings and numbers, but array items can also be objects, functions or even arrays.
 - You can have different data types in the same array.

PHP Forms Handling And Superglobals

► PHP - A Simple HTML Form

- The PHP superglobals `$_GET` and `$_POST` are used to collect form-data.
- Both `GET` and `POST` create an array (e.g. `array( key1 => value1, key2 => value2, key3 => value3, ...)`).
- Both `GET` and `POST` are treated as `$_GET` and `$_POST`. These are superglobals, which means that they are always accessible
- `$_GET` is an array of variables passed to the current script via the URL query parameters.
- `$_POST` is an array of variables passed to the current script via the HTTP POST method.

► PHP Superglobals

- PHP Superglobals are built-in variables that are always accessible in all scopes!
- Some predefined variables in PHP are "superglobals", which means that they are always accessible, regardless of scope - and you can access them from any function, class or file without having to do anything special.

PHP Forms Handling And Superglobals

► The PHP superglobal variables are:

- **`$GLOBALS`** - An array that contains references to all global variables of the script
- **`$_SERVER`** - Holds information about the web server including headers, paths, and script locations
- **`$_REQUEST`** - An array containing data from `$_GET`, `$_POST`, and `$_COOKIE` superglobals
- **`$_POST`** - An array of variables received via the HTTP POST method
- **`$_GET`** - An array of variables received via the HTTP GET method
- **`$_FILES`** - An array of items uploaded to the current script via the HTTP POST method (filename, type, size)
- **`$_ENV`** - Holds environment variables passed to the current script
- **`$_COOKIE`** - An array of variables passed to the current script via HTTP Cookies
- **`$_SESSION`** - An array of session variables

PHP Sessions, Cookies and Authentication Basics

► What is a PHP Session?

- Session variables hold information about one single user, and are available through all pages in one application.

► PHP Session Functions

- `session_start()` - Starts a new session
- `$_SESSION` - Stores and access session variables
- `unset()` - Removes a specific session variable (e.g `unset($_SESSION["favcolor"])`)
- `session_destroy()` - Destroys all data associated with the current session
- `session_unset()` - Frees all session variables

PHP Sessions, Cookies and Authentication Basics

► What is a Cookie?

- A cookie is a small text file that the server stores in the user's web browser. The cookie is then sent along with each HTTP request to the server.
- Cookies are ideal for storing information that can be retrieved later.
- Examples: (Username, Maintain login status, Shopping cart contents, Language preferences, Track user activity)

► Create Cookies With PHP

- A cookie is created with the `setcookie()` function.

► Syntax

- `setcookie(name, value, expire, path, domain, secure, httponly);`

PHP Sessions, Cookies and Authentication Basics

- ▶ Only the *name* parameter is required. All other parameters are optional.
 - **name** - Required. Specifies the name of the cookie
 - **value** - Optional. Specifies the value of the cookie
 - **expire** - Optional. Specifies when the cookie expires
 - **path** - Optional. Specifies the server path of the cookie
 - **domain** - Optional. Specifies the domain name of the cookie
 - **secure** - Optional. Specifies whether or not the cookie should only be transmitted over a secure HTTPS connection
 - **httponly** - Optional. If set to TRUE the cookie will be accessible only through the HTTP protocol

PHP CRUD Operations

► PHP Connect to MySQL

- There are two ways of working with PHP and MySQL.
- MySQLi (procedural)
- MySQLi (object-oriented)

PHP CRUD Operations

- ▶ Create a MySQL Database Using MySQLi
 - The CREATE DATABASE statement is used to create a database in MySQL.
 - MySQLi (procedural)
 - MySQLi (object-oriented)

PHP CRUD Operations

► Create a MySQL Table Using MySQLi

- The CREATE TABLE statement is used to create a table in MySQL.
- MySQLi (procedural)
- MySQLi (object-oriented)

PHP CRUD Operations

- ▶ Insert Data Into MySQL Using MySQLi
 - After a database and a table have been created, we can start adding data in them.
 - MySQLi (procedural)
 - MySQLi (object-oriented)
- ▶ Here are some syntax rules to follow:
 - The SQL query must be quoted in PHP
 - String values inside the SQL query must be quoted
 - Numeric values must not be quoted
 - The word NULL must not be quoted
- ▶ Insert Multiple Records Into MySQL Using MySQLi
 - Multiple SQL statements must be executed with the `multi_query()` function.

PHP CRUD Operations

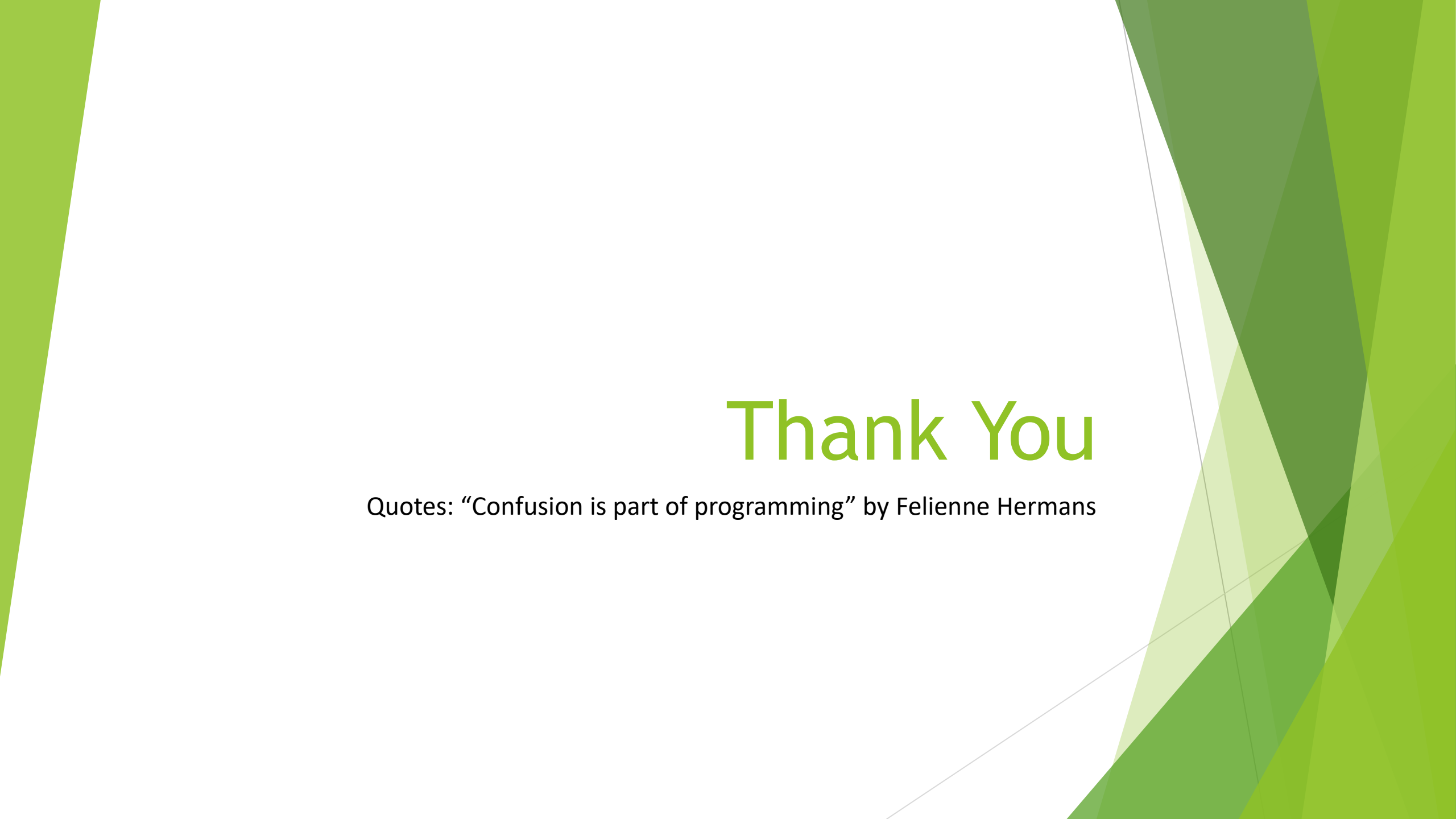
- ▶ Select Data With MySQLi
 - The SELECT statement is used to select data from one or more tables.
 - MySQLi (procedural)
 - MySQLi (object-oriented)

PHP CRUD Operations

- ▶ Update Data In a MySQL Table Using MySQLi
 - The UPDATE statement is used to update existing records in a table
 - MySQLi (procedural)
 - MySQLi (object-oriented)

PHP CRUD Operations

- ▶ Delete Data From a MySQL Table Using MySQLi
 - The DELETE statement is used to delete records from a table
 - MySQLi (procedural)
 - MySQLi (object-oriented)

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Thank You

Quotes: “Confusion is part of programming” by Felienne Hermans